

M.LOKESH CHOWDARY

AP24110011966

```
#include <stdio.h> #include <stdlib.h> #include <string.h>

#define MAX 100

// Stack structure typedef struct { char items[MAX][50]; int top; } Stack;

// Stack functions void init(Stack *s) { s->top = -1; }

int isFull(Stack *s) { return s->top == MAX - 1; }

int isEmpty(Stack *s) { return s->top == -1; }

void push(Stack *s, char *str) { if (isFull(s)) { printf("Stack is full!\n"); return; } strcpy(s->items[++s->top], str); }

char* pop(Stack *s) { if (isEmpty(s)) { return NULL; } return s->items[s->top--]; }

int main() { Stack undo, redo; init(&undo); init(&redo);

int choice;
char action[50];

while (1) {
    printf("\n--- Undo/Redo Simulation ---\n");
    printf("1. Perform Action\n");
    printf("2. Undo\n");
    printf("3. Redo\n");
    printf("4. View Stacks\n");
    printf("5. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter action (text): ");
            scanf("%[^\n]s", action);
            push(&undo, action);
```

```
// Clear redo stack
init(&redo);
printf("Action performed.\n");
break;
```

case 2:

```
if (!isEmpty(&undo)) {
    char *undone = pop(&undo);
    push(&redo, undone);
    printf("Undo: %s\n", undone);
} else {
    printf("Nothing to undo.\n");
}
break;
```

case 3:

```
if (!isEmpty(&redo)) {
    char *redone = pop(&redo);
    push(&undo, redone);
    printf("Redo: %s\n", redone);
} else {
    printf("Nothing to redo.\n");
}
break;
```

case 4:

```
printf("\nUndo Stack: ");
if (isEmpty(&undo)) printf("empty");
for (int i = 0; i <= undo.top; i++)
    printf("%s ", undo.items[i]);

printf("\nRedo Stack: ");
if (isEmpty(&redo)) printf("empty");
for (int i = 0; i <= redo.top; i++)
    printf("%s ", redo.items[i]);
printf("\n");
break;
```

case 5:

```
exit(0);
```

```

        default:
            printf("Invalid choice.\n");
        }
    }

return 0;

}

#include <stdio.h> #include <stdlib.h> #include <string.h>

#define MAX 100

// Stack structure typedef struct { char items[MAX][50]; int top; } Stack;

// Stack functions void init(Stack *s) { s->top = -1; }

int isFull(Stack *s) { return s->top == MAX - 1; }

int isEmpty(Stack *s) { return s->top == -1; }

void push(Stack *s, char *str) { if (isFull(s)) { printf("Stack is full!\n"); return; } strcpy(s->items[++s->top], str); }

char* pop(Stack *s) { if (isEmpty(s)) { return NULL; } return s->items[s->top--]; }

int main() { Stack undo, redo; init(&undo); init(&redo);

int choice;
char action[50];

while (1) {
    printf("\n--- Undo/Redo Simulation ---\n");
    printf("1. Perform Action\n");
    printf("2. Undo\n");
    printf("3. Redo\n");
    printf("4. View Stacks\n");
    printf("5. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:

```

```
printf("Enter action (text): ");
scanf(" %[^\\n]s", action);
push(&undo, action);
// Clear redo stack
init(&redo);
printf("Action performed.\\n");
break;
```

case 2:

```
if (!isEmpty(&undo)) {
    char *undone = pop(&undo);
    push(&redo, undone);
    printf("Undo: %s\\n", undone);
} else {
    printf("Nothing to undo.\\n");
}
break;
```

case 3:

```
if (!isEmpty(&redo)) {
    char *redone = pop(&redo);
    push(&undo, redone);
    printf("Redo: %s\\n", redone);
} else {
    printf("Nothing to redo.\\n");
}
break;
```

case 4:

```
printf("\\nUndo Stack: ");
if (isEmpty(&undo)) printf("empty");
for (int i = 0; i <= undo.top; i++)
    printf("%s ", undo.items[i]);

printf("\\nRedo Stack: ");
if (isEmpty(&redo)) printf("empty");
for (int i = 0; i <= redo.top; i++)
    printf("%s ", redo.items[i]);
printf("\\n");
break;
```

```

case 5:
    exit(0);

default:
    printf("Invalid choice.\n");
}
}

return 0;

}

```

## OUTPUT

--- Undo/Redo Simulation ---

1. Perform Action
2. Undo
3. Redo
4. View Stacks
5. Exit Enter choice: 1 Enter action (text): helo Action performed.

--- Undo/Redo Simulation ---

1. Perform Action
2. Undo
3. Redo
4. View Stacks
5. Exit Enter choice: 2 Undo: helo

--- Undo/Redo Simulation ---

1. Perform Action
2. Undo
3. Redo
4. View Stacks
5. Exit Enter choice: 3 Redo: helo

--- Undo/Redo Simulation ---

1. Perform Action
2. Undo
3. Redo
4. View Stacks
5. Exit Enter choice: 4

Undo Stack: helo Redo Stack: empty

--- Undo/Redo Simulation ---

1. Perform Action
2. Undo
3. Redo
4. View Stacks
5. Exit Enter choice: 5

=== Code Execution Successful ===